Cryptocurrency Market Prediction

DRW Crypto Market Prediction Kaggle Challenge

Andre Barle

Problem Statement

Source: https://en.wikipedia.org/wiki/Pearson_correlation_coefficient

- DRW launched a Kaggle competition aimed at solving the crypto market
- Provided with market snapshots of a particular cryptocurrency or cryptocurrency index
- All **data provided is anonymized** in terms of what we are predicting for and the features are mostly anonymous
- We have **895 features** and **525,887 timestamps** of a cryptocurrency from **2023-03-01–2024-02-29**
- Need to create a model whose predictions have the highest Pearson Correlation with unseen test data

$$\rho_{X,Y} = \frac{\mathbb{E}[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

Pearson Correlation Formula:

Computes the covariance of two variables divided by the product of their standard deviations

We are testing predictions vs actual in terms of Pearson correlation

Data and Task

Source: https://www.kaggle.com/competitions/drw-crypto-market-prediction/overview

- **timestamp**: The timestamp index representing the minute associated with each row.
- **bid_qty**: The total quantity buyers are willing to purchase at the best (highest) bid price at the given timestamp.
- **ask_qty**: The total quantity sellers are offering to sell at the best (lowest) ask price at the given timestamp.
- **buy_qty**: The total trading quantity executed at the best ask price during the given minute.
- **sell_qty**: The total trading quantity executed at the best bid price during the given minute.
- **volume**: The total traded volume during the minute.
- **X_{1,...,780}**: A set of anonymized market features derived from proprietary data sources.
- **label**: The target variable representing the anonymized market price movement to be predicted.

- Must make a model whose predictions are highly correlated to the test set actual labels via Pearson correlation
  - Means the model must **generalize well on unseen data**
  - Must **avoid overfitting**
  - Time sensitive splits need to ensure **no future peeking**

<class 'pandas.core.frame.DataFrame'>

timestamp	bid_qty	ask_qty	buy_qty	sell_qty	volume	X1	X2	X3	X4	X5	...	X882	X883	X884	X885	X886	X887	X888	X889
2023-03-01 00:00:00	15.283	8.425	176.405	44.984	221.389	0.121263	-0.417690	0.005399	0.125948	0.058359	...	1.925423	1.847943	0.005676	0.190791	0.369691	0.377630	0.210153	0.159183
2023-03-01 00:01:00	38.590	2.336	525.846	321.950	847.796	0.302841	-0.049576	0.356667	0.481087	0.237954	...	1.928569	1.849468	0.005227	0.184660	0.363642	0.374515	0.209573	0.158963
2023-03-01 00:02:00	0.442	60.250	159.227	136.369	295.596	0.167462	-0.291212	0.083138	0.206881	0.101727	...	1.928047	1.849282	0.004796	0.178719	0.357689	0.371424	0.208993	0.158744
2023-03-01 00:03:00	4.865	21.016	335.742	124.963	460.705	0.072944	-0.436590	-0.102483	0.017551	0.007149	...	1.928621	1.849608	0.004398	0.172967	0.351832	0.368358	0.208416	0.158524
2023-03-01 00:04:00	27.158	3.451	98.411	44.407	142.818	0.173820	-0.213489	0.096067	0.215709	0.107133	...	1.927084	1.848950	0.004008	0.167391	0.346066	0.365314	0.207839	0.158304
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2024-02-29 23:55:00	4.163	6.805	39.037	55.351	94.388	0.020155	0.076565	0.228994	0.288856	0.151634	...	3.219345	3.340686	0.008679	0.224656	0.401595	0.393726	0.212651	0.136494
2024-02-29 23:56:00	2.290	4.058	110.201	67.171	177.372	0.016262	0.062527	0.214072	0.276463	0.146521	...	3.216719	3.339353	0.007928	0.217422	0.395019	0.390476	0.212063	0.136305
2024-02-29 23:57:00	5.237	3.640	70.499	30.753	101.252	0.045407	0.109834	0.263577	0.329266	0.174214	...	3.213444	3.337740	0.007243	0.210421	0.388549	0.387252	0.211477	0.136117
2024-02-29 23:58:00	5.731	4.901	22.365	52.195	74.560	0.124783	0.244168	0.408704	0.480016	0.251493	...	3.209945	3.336030	0.006608	0.203642	0.382184	0.384054	0.210892	0.135928
2024-02-29 23:59:00	3.925	3.865	86.585	217.137	303.722	0.368659	0.665382	0.867538	0.951903	0.491276	...	3.208415	3.335166	0.006072	0.197096	0.375931	0.380886	0.210310	0.135741

525887 rows × 896 columns

Methodology

Brief Overview of steps

- Feature engineering for **imbalances, aggressiveness, and interactions** with the market
  - Dropping any constant or empty features
  - Ensuring no future peeking
- Grid search pipeline of **Ridge, Lasso, and ElasticNet**
  - **SimpleImputer** to handle missing values
  - **StandardScaler** to scale the data
- Used **k-fold cross validation** for 4 folds of the data
  - For Lasso Alpha = 0.01
  - For Ridge Alpha = 15
  - For ElasticNet
    - Alpha = 0.01
    - L1 = 0.6
  - Select MSE that produces the best Pearson r score per each fold

Hold out 15% of data for testing for final out of fold prediction

Each CV fold does not future peek, generates predictions within the fold, and updates the model weights to the next fold

The final test is on the holdout data

Lasso Algorithm

Source: https://scikit-learn.org/stable/modules/linear_model.html#lasso

- Ordinary Regression minimizes the sum of squared residuals
  - Weights can become very large
  - Uses all features to minimize error
- Lasso applies an L1 regularization term to shrink coefficients that are not useful
  - Pushes **coefficients to 0**
  - Performs **automatic feature selection**
  - Typically **increases bias and lowers variance**

$$\min_w \frac{1}{2n_{\text{samples}}} ||Xw - y||_2^2 + \alpha||w||_1$$

Xw - y = residuals vector, predictions - target

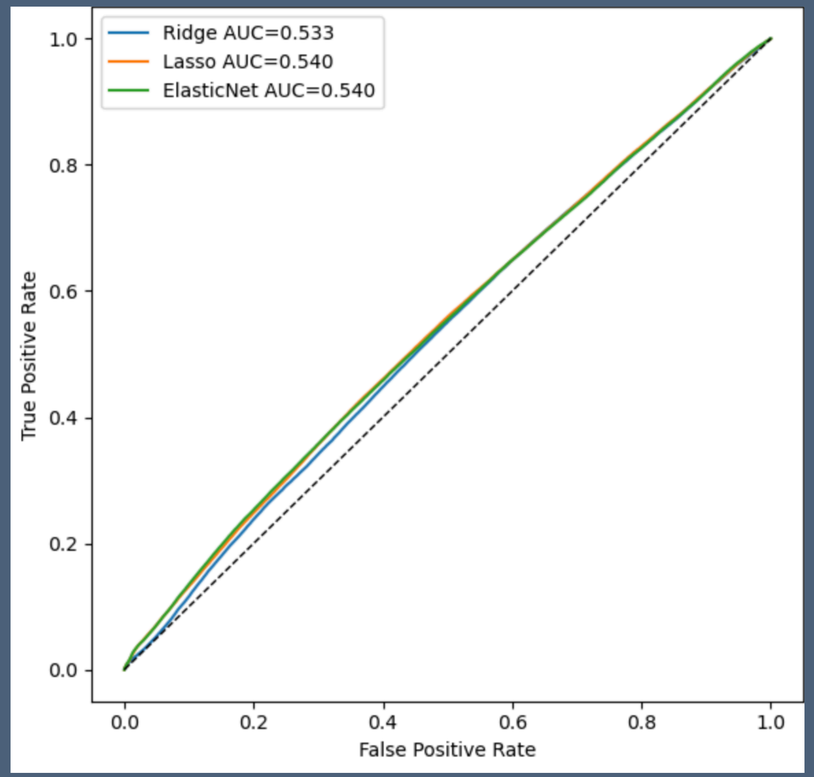
1/2n_samples = scaling the mean squared error

alpha||w||1 = L1 regularization term where larger alpha reduces the coefficients of less useful features towards 0

Results

ROC Curve

ROC Curve (out of fold) predictions

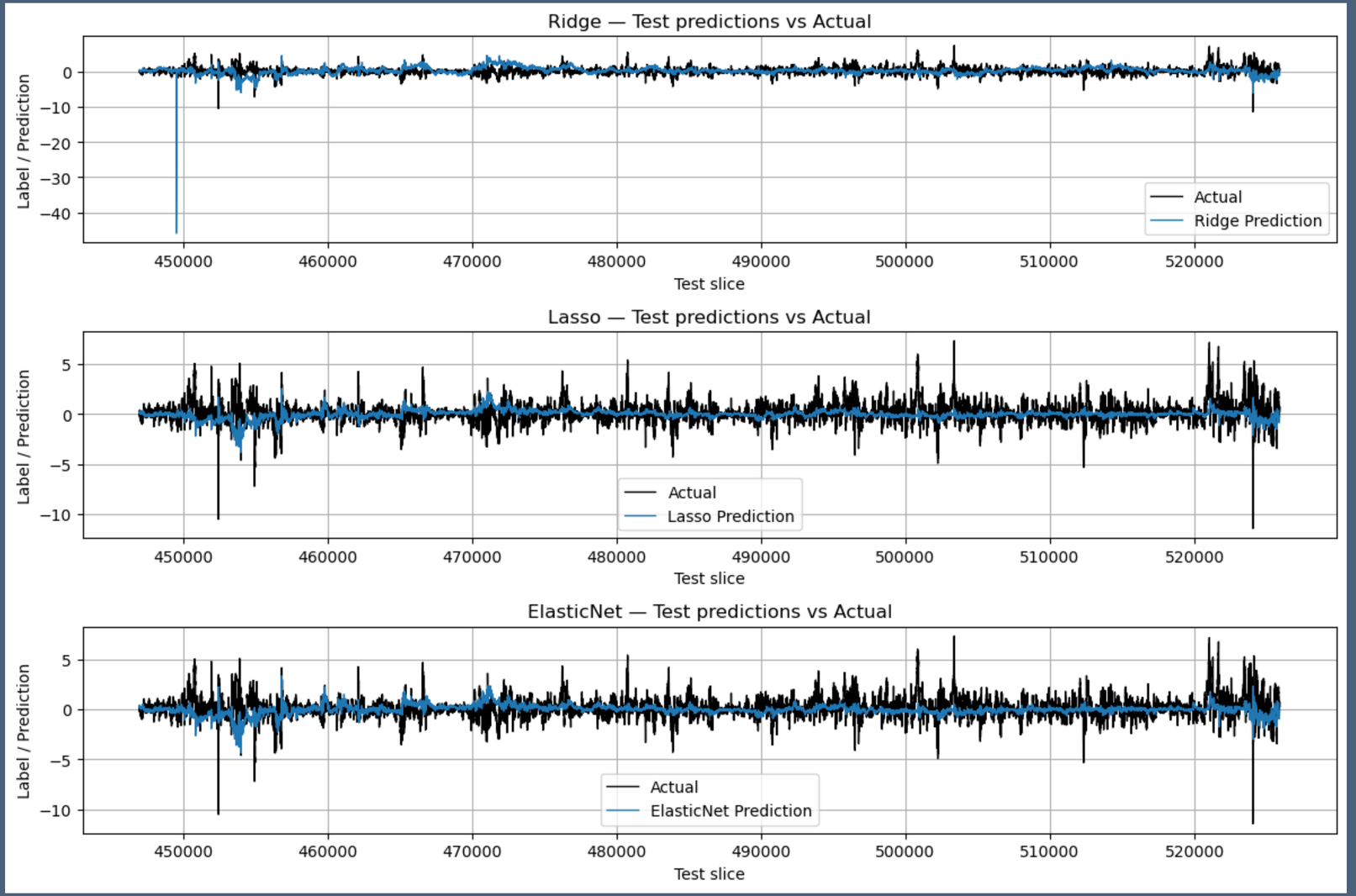


- Neural Networks **did not work** for this task as they overfit or **do not converge**
- Hidden Markov Chains which normally work on financial data **did not work better than any form of regression**
- In the end, Ridge, Lasso, and ElasticNet worked the best, with **Lasso winning** based on Pearson score

Results

Pearson r scores:

- 1: Lasso r: **0.128924**
- 2: ElasticNet r: **0.120861**
- 3: Ridge r: **0.044380**



Rationale

- **Lasso works better** than Ridge for this task because it is effectively **able to perform feature selection**
  - **Ridge** reduces coefficients via **L2 regularization proportionally** and keeps all predictors which can make it sensitive to noise in real applications
  - **Lasso selects strong features and lowers others to 0** weight, effectively capturing some weak signals in the data and making them stronger
  - **ElasticNet** performs as a **hybrid approach mixing L1 and L2** regularization and performed only slightly worse than Lasso

L2 regularization penalizes large coefficients heavily and reduces them close to 0

L1 regularization penalizes coefficients linearly and reduces them to 0 if they are not useful

Discussion

Next Steps

Lesson Learned: Simple models can work much better on real data than complex models

Improvements in the model can be made by feature selection and feature engineering

Removing noisy data and creating meaningful features

- Predicting any market outcome is very difficult, so having at least some correlation with the outcome of the market is valuable
- The problem will continue to be unsolved, the competition has no score higher than **0.13959** on the private testing round

Thank you